

Constructor in java

By now, you have understood methods.

You will now understand how are objects created.

Whenever we create a new object, we need to assign values to the instance variables individually for each object.

```
public static void main(String args[]) {  
    Customer customer1 = new Customer();  
    customer1.customerId = "C101";  
    customer1.customerName = "Jack";  
    customer1.contactNumber = 9870345687L;  
    customer1.address = "Manor Farm Barns, Fox Road, Norwich, England";  
    Customer customer2 = new Customer();  
    customer2.customerId = "C102";  
    customer2.customerName = "Jane";  
    customer2.contactNumber = 7870098687L;  
    customer2.address = "Morningside Road, Edinburgh, Scotland";  
}
```

Consider the scenario where we have many customers. We would have to initialize each attribute of each object as shown above. This process would be much simpler if we had some other way of initializing all the members of an object while creating the object. This can be achieved by **constructors**.

A **constructor** in Java is a special method that is used to initialize class variables at the time of object creation.

Each time an object is created using the new() keyword, a constructor is called. A constructor can be created by the programmer. If the developer does not create any constructor, then, Java provides a default constructor.

In the previous examples, we had not created any constructor, but Java provided a default constructor for every class.

Constructors have the same name as that of the class and does not have a return type.

Syntax:

```
<access modifier> <name> (<parameters>){  
    //body  
}
```

```
public Customer(){  
}
```

A constructor with no arguments is known as a **parameterless constructor**. If you don't define a constructor in a class, then Java creates a default parameterless constructor and initializes the default values to the class variables based on the data type.

```
class Customer {  
    public String customerId;  
    public String customerName;  
    public long contactNumber;  
    public String address;  
}  
public class Tester {  
    public static void main(String args[]) {  
        Customer customer1 = new Customer();  
        System.out.println(customer1.customerId);  
        System.out.println(customer1.customerName);  
        System.out.println(customer1.contactNumber);  
        System.out.println(customer1.address);  
    }  
}
```

Here, the default parameterless constructor is called every time a new object is created and the default values to the member variables are assigned.

```
null  
null  
0  
null
```

You can also create **parameterless** constructor in a class. In this case, Java does not create a separate default constructor.

```
class Customer {
    public String customerId;
    public String customerName;
    public long contactNumber;
    public String address;
    public Customer() {
        System.out.println("Constructor called");
    }
}

public class Tester {
    public static void main(String args[]) {
        Customer customer1 = new Customer();
        System.out.println(customer1.customerId);
        System.out.println(customer1.customerName);
        System.out.println(customer1.contactNumber);
        System.out.println(customer1.address);
        Customer customer2 = new Customer();
    }
}
```

```
Constructor called  
null  
null  
0  
null  
Constructor called
```

Like any other method, a constructor can also accept parameters. Generally, these are the values that need to be assigned to the instance variables of the class for that object.

```
class Customer {  
    public String customerId;  
    public String customerName;  
    public long contactNumber;  
    public String address;  
    Customer(String cId, String cName, long contact, String add) {  
        customerId = cId;  
        customerName = cName;  
        contactNumber = contact;  
        address = add;  
    }  
}
```

```
public class Tester {  
    public static void main(String args[]) {  
        Customer customer1 = new Customer("C103", "Jacob", 5648394590L,  
                                           "13th Street, New York");  
        System.out.println(customer1.customerId);  
        System.out.println(customer1.customerName);  
        System.out.println(customer1.contactNumber);  
        System.out.println(customer1.address);  
    }  
}
```

```
C103  
Jacob  
5648394590  
13th Street, New York
```

Problem Statement Execute the below code to understand the working of constructors.

```
class Customer {
    public String customerId;
    public String customerName;
    public long contactNumber;
    public String address;

    /*public Customer() {
        System.out.println("Parameterless constructor called");
    }*/

    public Customer(String cId, String cName, long contact, String add) {
        System.out.println("Parameterized constructor called");
        customerId = cId;
        customerName = cName;
        contactNumber = contact;
        address = add;
    }
}

class Tester{
    public static void main(String args[]) {
        //the below line will lead to an error as there is no default constructor
        Customer customer2 = new Customer();
        //if a parameterized constructor is created in a class, java doesn't create a default constructor
        //uncomment the parameterless constructor and then execute the code

        //the below code leads to an error as the parameters accepted by the constructor does not match the parameters passed
        Customer customer1 = new Customer("C103", "Jacob", "13th Street, New York", 5648394590L);
        //the order and type of the parameters passed for a parameterized constructor should match the definition properly
    }
}
```

A class can have multiple constructors to initialize different members. Based on the arguments passed, the respective constructor is called.

```
class Customer {
    public String customerId;
    public String customerName;
    public long contactNumber;
    public String address;
    public Customer() {
        System.out.println("Parameterless constructor called");
    }
    public Customer(String cId, String cName, long contact, String add) {
        System.out.println("Parameterized constructor called");
        customerId = cId;
        customerName = cName;
        contactNumber = contact;
        address = add;
    }
}

public class Tester {
    public static void main(String args[]) {
        Customer customer1 = new Customer("C103", "Jacob", 5648394590L,
                                           "13th Street, New York");
        Customer customer2 = new Customer();
    }
}
```

What will be the output of the below code?

```
class Demo {  
    public int value = 20;  
  
    Demo() {  
        value = 40;  
    }  
}  
  
public class Tester {  
    public static void main(String args[]) {  
        Demo demo = new Demo();  
        System.out.println(demo.value);  
    }  
}
```

0

20

40

Compilation error: variable 'value' is not declared

What will be the output of the below code?

```
class Demo {  
    public int var1 = 20;  
    public int var2 = 40;  
  
    Demo(int v1, int v2) {  
        var1 = v1;  
        var2 = v2;  
    }  
}  
  
public class Tester {  
    public static void main(String args[]) {  
        Demo demo = new Demo();  
        System.out.println(demo.var1);  
        System.out.println(demo.var2);  
    }  
}
```

20 40

0 0

40 20

compilation error: class Tester has no default constructor

Consider the below Circle class.

```
public class Circle {  
    public double radius;  
  
    Circle(double r) {  
        radius = r;  
    }  
}
```

Which of the following options will create an object for the class?

Circle c = new Circle(10.5)

Circle = new Circle(10.5)

Circle c = new Circle()

Circle c = new Circle(5.4, 10)

Which of the following is/are true about constructors in Java?

- The name of the constructor should be same as the class name.
- If you don't define a constructor for a class, a default parameterless constructor is automatically provided by the compiler.
- The default constructor initializes all the instance variables to default values of the data types.
- A constructor cannot have an access modifier.

Option 40 is Correct

Explanation :

When the object of Demo class is created, constructor gets invoked. So, the value of the instance variable 'value' is 40.

Option compilation error: class Tester has no default constructor is Correct

Explanation :

If a parameterized constructor is created by the developer, Java does not supply a default constructor. Since, there is no parameterless constructor present, there will be a compilation error.

Option Circle c = new Circle(10.5) is Correct

Explanation :

This is the correct way to create an object.

Option The name of the constructor should be same as the class name. is Correct

Explanation :

The name of the constructor is always the same as the class name.

Option If you don't define a constructor for a class, a default parameterless constructor is automatically provided by the compiler. is C

Explanation :

If you don't define a constructor for a class, a default parameterless constructor is automatically provided by the compiler.

Option The default constructor initializes all the instance variables to default values of the data types. is Correct

Explanation :

The default constructor initializes all the instance variables to default values of the data types.